



# Combinational Logic



# Combinational Circuit

---

- A combinational circuit consists of logic gates whose outputs at any time are determined from the present combination of inputs.
- Does not use any memory
- It can have  $n$  number of inputs &  $m$  number of outputs.
- Examples: Adders, Decoders, MUX, Code converters etc



# Combinational Logic Circuit

```
graph TD; A[Combinational Logic Circuit] --> B[Arithmetic & Logical Functions]; A --> C[Data Transmission]; A --> D[Code Converters]; B --> B1[Adders<br/>Subtractors<br/>Comparitors<br/>PLD's]; C --> C1[Multiplexers<br/>Demultiplexers<br/>Encoders<br/>Decoders]; D --> D1[Binary<br/>BCD<br/>7-segment];
```

Arithmetic &  
Logical Functions

Adders  
Subtractors  
Comparitors  
PLD's

Data  
Transmission

Multiplexers  
Demultiplexers  
Encoders  
Decoders

Code  
Converters

Binary  
BCD  
7-segment

# Analysis Procedure

---

- Determine the function, that the circuit implemented.
- Logic Diagram
  - Boolean Function(s)
  - Truth Table
  - Explanation of the circuit



# Design Procedure

---

1. Specification of Circuit
2. Determine required number of inputs and outputs
3. Derive the truth table
4. Get simplified Boolean function for each output as a function of input variables.
5. Draw the logic diagram
  - Verify the correctness of design



# Design Procedure - Example

---

- Specification of Circuit

- Design a circuit that converts BCD code into Excess-3 Code.

- **Step-I:**

- Determine the number of inputs
  - Determine the number of outputs



# Design Procedure - Example

---

- Specification of Circuit

- BCD Code:

- Binary Code that represent decimal digits
- How many decimal digits ???
- How many bits required to do represent these digits ???
- Thus the number of inputs = 4



# BCD

| Decimal Digit | A | B | C | D |
|---------------|---|---|---|---|
| 0             | 0 | 0 | 0 | 0 |
| 1             | 0 | 0 | 0 | 1 |
| 2             | 0 | 0 | 1 | 0 |
| 3             | 0 | 0 | 1 | 1 |
| 4             | 0 | 1 | 0 | 0 |
| 5             | 0 | 1 | 0 | 1 |
| 6             | 0 | 1 | 1 | 0 |
| 7             | 0 | 1 | 1 | 1 |
| 8             | 1 | 0 | 0 | 0 |
| 9             | 1 | 0 | 0 | 1 |

## Excess-3

| Decimal Digit | w | x | y | z |
|---------------|---|---|---|---|
| 0             | 0 | 0 | 1 | 1 |
| 1             | 0 | 1 | 0 | 0 |
| 2             | 0 | 1 | 0 | 1 |
| 3             | 0 | 1 | 1 | 0 |
| 4             | 0 | 1 | 1 | 1 |
| 5             | 1 | 0 | 0 | 0 |
| 6             | 1 | 0 | 0 | 1 |
| 7             | 1 | 0 | 1 | 0 |
| 8             | 1 | 0 | 1 | 1 |
| 9             | 1 | 1 | 0 | 0 |

# Truth Table of Design Example

| Input BCD |   |   |   |  | Output Excess-3 |   |   |   |
|-----------|---|---|---|--|-----------------|---|---|---|
| A         | B | C | D |  | w               | x | y | z |
| 0         | 0 | 0 | 0 |  |                 |   |   | 1 |
| 0         | 0 | 0 | 1 |  |                 |   |   | 0 |
| 0         | 0 | 1 | 0 |  |                 |   |   | 1 |
| 0         | 0 | 1 | 1 |  |                 |   |   | 0 |
| 0         | 1 | 0 | 0 |  |                 |   |   | 1 |
| 0         | 1 | 0 | 1 |  |                 |   |   | 0 |
| 0         | 1 | 1 | 0 |  |                 |   |   | 1 |
| 0         | 1 | 1 | 1 |  |                 |   |   | 0 |
| 1         | 0 | 0 | 0 |  |                 |   |   | 1 |
| 1         | 0 | 0 | 1 |  |                 |   |   | 0 |

# Truth Table of Design Example

| Input BCD |   |   |   |  | Output Excess-3 |   |   |   |
|-----------|---|---|---|--|-----------------|---|---|---|
| A         | B | C | D |  | w               | x | y | z |
| 0         | 0 | 0 | 0 |  | 0               | 0 | 1 | 1 |
| 0         | 0 | 0 | 1 |  | 0               | 1 | 0 | 0 |
| 0         | 0 | 1 | 0 |  | 0               | 1 | 0 | 1 |
| 0         | 0 | 1 | 1 |  | 0               | 1 | 1 | 0 |
| 0         | 1 | 0 | 0 |  | 0               | 1 | 1 | 1 |
| 0         | 1 | 0 | 1 |  | 1               | 0 | 0 | 0 |
| 0         | 1 | 1 | 0 |  | 1               | 0 | 0 | 1 |
| 0         | 1 | 1 | 1 |  | 1               | 0 | 1 | 0 |
| 1         | 0 | 0 | 0 |  | 1               | 0 | 1 | 1 |
| 1         | 0 | 0 | 1 |  | 1               | 1 | 0 | 0 |

## Getting simplified Boolean Function of the Design Example

| Input BCD |   |   |   | Output Excess-3 |   |   |   |
|-----------|---|---|---|-----------------|---|---|---|
| A         | B | C | D | w               | x | y | z |
| 0         | 0 | 0 | 0 | 0               | 0 | 1 | 1 |
| 0         | 0 | 0 | 1 | 0               | 1 | 0 | 0 |
| 0         | 0 | 1 | 0 | 0               | 1 | 0 | 1 |
| 0         | 0 | 1 | 1 | 0               | 1 | 1 | 0 |
| 0         | 1 | 0 | 0 | 0               | 1 | 1 | 1 |
| 0         | 1 | 0 | 1 | 1               | 0 | 0 | 0 |
| 0         | 1 | 1 | 0 | 1               | 0 | 0 | 1 |
| 0         | 1 | 1 | 1 | 1               | 0 | 1 | 0 |
| 1         | 0 | 0 | 0 | 1               | 0 | 1 | 1 |
| 1         | 0 | 0 | 1 | 1               | 1 | 0 | 0 |

| AB \ CD | CD |    |    |    |
|---------|----|----|----|----|
|         | 00 | 01 | 11 | 10 |
| 00      |    |    |    |    |
| 01      |    |    |    |    |
| 11      |    |    |    |    |
| 10      |    |    |    |    |

## Getting simplified Boolean Function of the Design Example

| Input BCD |   |   |   | Output Excess-3 |   |   |   |
|-----------|---|---|---|-----------------|---|---|---|
| A         | B | C | D | w               | x | y | z |
| 0         | 0 | 0 | 0 |                 |   |   | 1 |
| 0         | 0 | 0 | 1 |                 |   |   | 0 |
| 0         | 0 | 1 | 0 |                 |   |   | 1 |
| 0         | 0 | 1 | 1 |                 |   |   | 0 |
| 0         | 1 | 0 | 0 |                 |   |   | 1 |
| 0         | 1 | 0 | 1 |                 |   |   | 0 |
| 0         | 1 | 1 | 0 |                 |   |   | 1 |
| 0         | 1 | 1 | 1 |                 |   |   | 0 |
| 1         | 0 | 0 | 0 |                 |   |   | 1 |
| 1         | 0 | 0 | 1 |                 |   |   | 0 |

| AB \ CD |    |    |    |    |
|---------|----|----|----|----|
|         | 00 | 01 | 11 | 10 |
| 00      | 1  |    |    | 1  |
| 01      | 1  |    |    | 1  |
| 11      | x  | x  | x  | x  |
| 10      | 1  |    | x  | x  |

$$z = D'$$

## Getting simplified Boolean Function of the Design Example

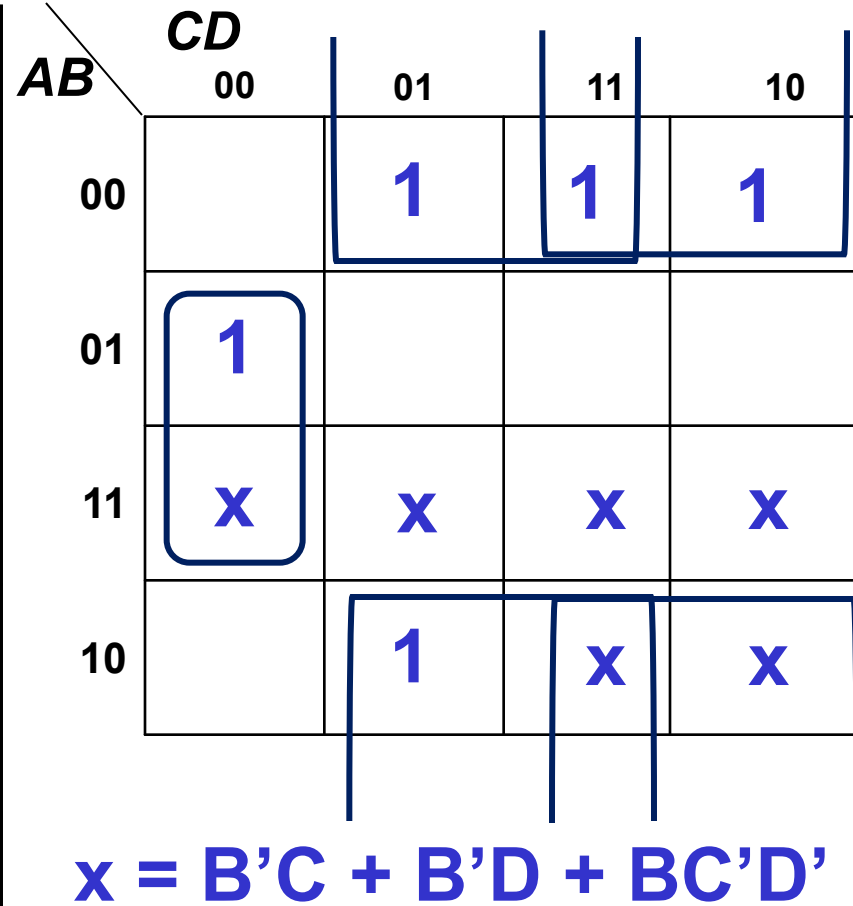
| Input BCD |   |   |   | Output Excess-3 |   |   |   |
|-----------|---|---|---|-----------------|---|---|---|
| A         | B | C | D | w               | x | y | z |
| 0         | 0 | 0 | 0 |                 |   | 1 |   |
| 0         | 0 | 0 | 1 |                 |   | 0 |   |
| 0         | 0 | 1 | 0 |                 |   | 0 |   |
| 0         | 0 | 1 | 1 |                 |   | 1 |   |
| 0         | 1 | 0 | 0 |                 |   | 1 |   |
| 0         | 1 | 0 | 1 |                 |   | 0 |   |
| 0         | 1 | 1 | 0 |                 |   | 0 |   |
| 0         | 1 | 1 | 1 |                 |   | 1 |   |
| 1         | 0 | 0 | 0 |                 |   | 1 |   |
| 1         | 0 | 0 | 1 |                 |   | 0 |   |

| AB \ CD |    |    |    |    |
|---------|----|----|----|----|
|         | 00 | 01 | 11 | 10 |
| 00      | 1  |    | 1  |    |
| 01      | 1  |    | 1  |    |
| 11      | x  | x  | x  | x  |
| 10      | 1  |    | x  | x  |

$$y = C'D' + CD$$

## Getting simplified Boolean Function of the Design Example

| Input BCD |   |   |   | Output Excess-3 |   |   |   |
|-----------|---|---|---|-----------------|---|---|---|
| A         | B | C | D | w               | x | y | z |
| 0         | 0 | 0 | 0 |                 | 0 |   |   |
| 0         | 0 | 0 | 1 |                 | 1 |   |   |
| 0         | 0 | 1 | 0 |                 | 1 |   |   |
| 0         | 0 | 1 | 1 |                 | 1 |   |   |
| 0         | 1 | 0 | 0 |                 | 1 |   |   |
| 0         | 1 | 0 | 1 |                 | 0 |   |   |
| 0         | 1 | 1 | 0 |                 | 0 |   |   |
| 0         | 1 | 1 | 1 |                 | 0 |   |   |
| 1         | 0 | 0 | 0 |                 | 0 |   |   |
| 1         | 0 | 0 | 1 |                 | 1 |   |   |



## Getting simplified Boolean Function of the Design Example

| Input BCD |   |   |   | Output Excess-3 |   |   |   |
|-----------|---|---|---|-----------------|---|---|---|
| A         | B | C | D | w               | x | y | z |
| 0         | 0 | 0 | 0 | 0               |   |   |   |
| 0         | 0 | 0 | 1 | 0               |   |   |   |
| 0         | 0 | 1 | 0 | 0               |   |   |   |
| 0         | 0 | 1 | 1 | 0               |   |   |   |
| 0         | 1 | 0 | 0 | 0               |   |   |   |
| 0         | 1 | 0 | 1 | 1               |   |   |   |
| 0         | 1 | 1 | 0 | 1               |   |   |   |
| 0         | 1 | 1 | 1 | 1               |   |   |   |
| 1         | 0 | 0 | 0 | 1               |   |   |   |
| 1         | 0 | 0 | 1 | 1               |   |   |   |

| AB \ CD | CD |    |    |    |
|---------|----|----|----|----|
|         | 00 | 01 | 11 | 10 |
| 00      |    |    |    |    |
| 01      |    | 1  | 1  | 1  |
| 11      | x  | x  | x  | x  |
| 10      | 1  | 1  | x  | x  |

$$w = A + BC + BD$$



# Simplified Boolean Function

---

$$\square z = D'$$

$$\square y = CD + C'D'$$

$\square$

$$\square x = B'C + B'D + BC'D'$$

$\square$

$$\square w = A + BC + BD = A + B(C+D)$$



---

$$\square z = D'$$

$$\square y = CD + C'D'$$

$$\square = CD + (C+D)'$$

$$\square x = B'C + B'D + BC'D'$$

$$\square = B'(C+D) + BC'D'$$

$$\square = B'(C+D) + B(C+D)'$$

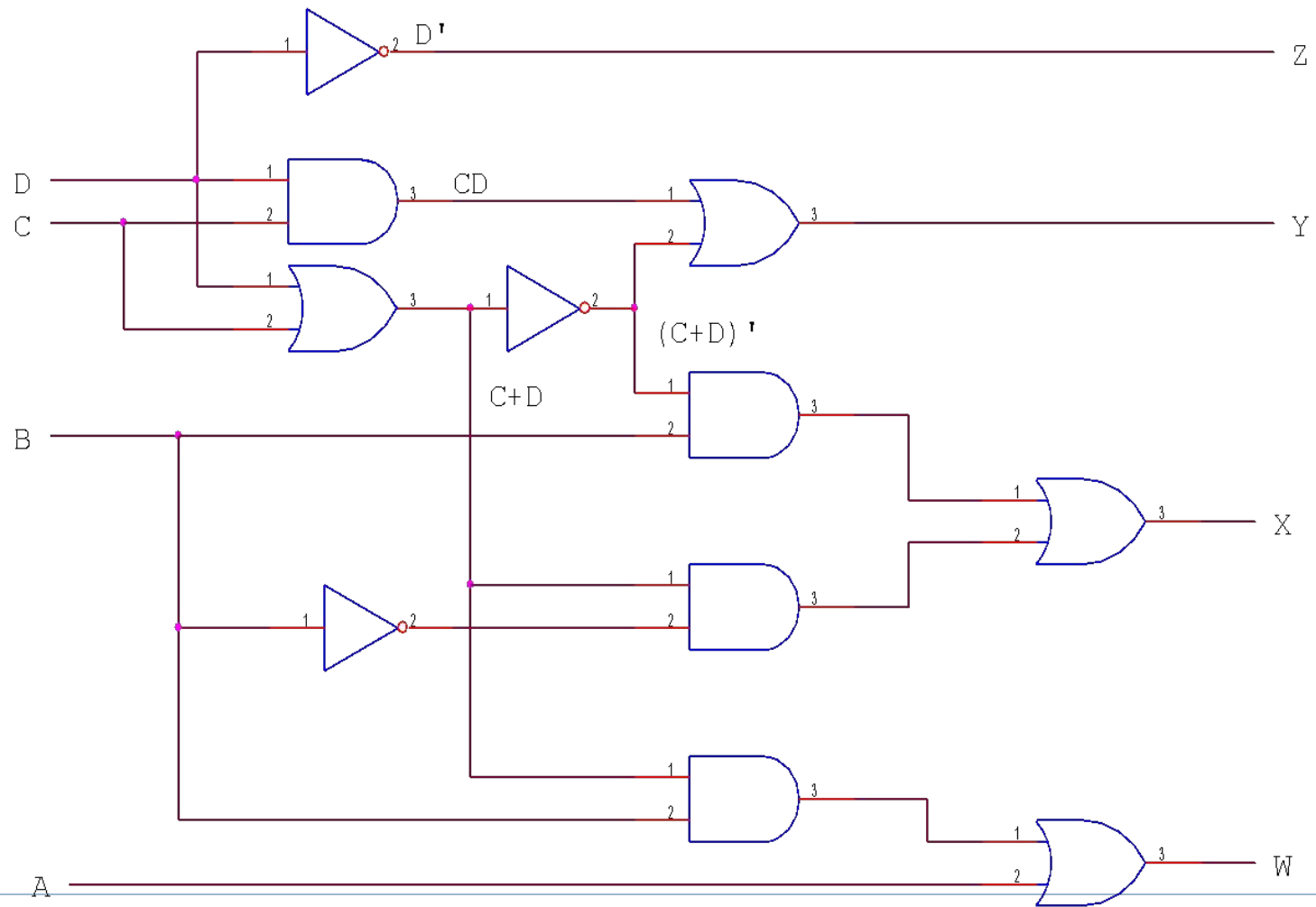
$$\square w = A + BC + BD$$

$$\square w = A + B(C+D)$$

---



# Example Design Procedure – Circuit Diagram



**Problem:** Design a combinational circuit with three inputs and one output. The output is 1 when the binary value of the inputs is less than 3. The output is 0 otherwise.

---

Step-1

Specification of Circuit.

Step-2

Determine Number of Inputs & Outputs

Step-3

Derive Truth Table

Step-4

Get simplified Boolean Equations

Step-5

Draw Logic Diagram & Verify



**Problem:** Design a combinational circuit with three inputs  $x$ ,  $y$ , and  $z$ , and three outputs  $A$ ,  $B$ , and  $C$ . When the binary input is 0, 1, 2, or 3, the binary output is one greater than the input. When the binary input is 4, 5, 6, or 7, the binary output is one less than the input.

---

### Step-1

Specification of Circuit.

### Step-2

Determine Number of Inputs & Outputs

### Step-3

Derive Truth Table

### Step-4

Get simplified Boolean Equations

### Step-5

Draw Logic Diagram & Verify



# Half and Full Adder

# Half Adder

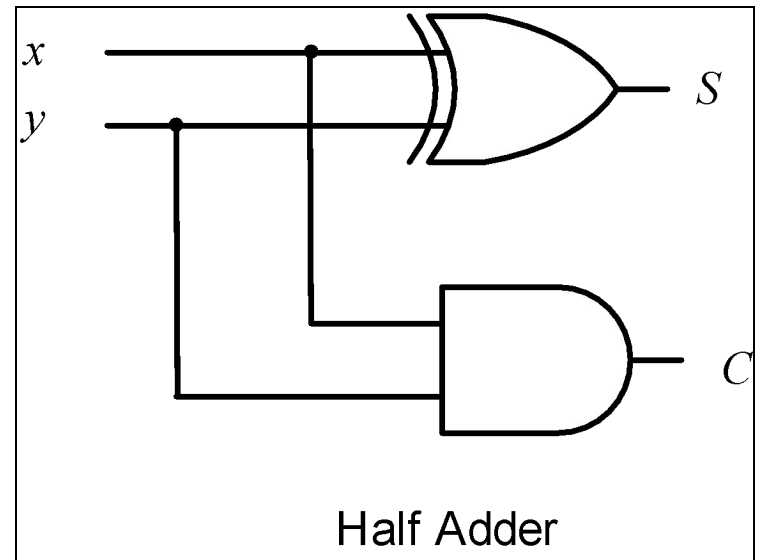
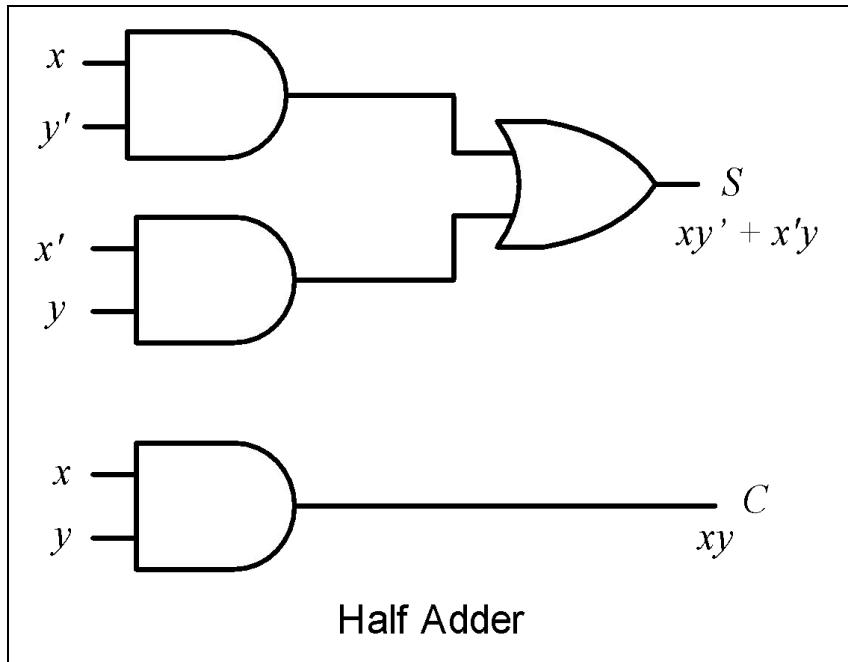
---

- A combinational circuit that performs the addition of two bits is called a half adder.

| <b><i>x</i></b> | <b><i>y</i></b> | <b><i>C</i></b> | <b><i>S</i></b> |
|-----------------|-----------------|-----------------|-----------------|
| 0               | 0               | 0               | 0               |
| 0               | 1               | 0               | 1               |
| 1               | 0               | 0               | 1               |
| 1               | 1               | 1               | 0               |



# Half Adder



# Full Adder

---

- A combinational circuit that performs the addition of three bits is called a FULL adder.



# Full Adder

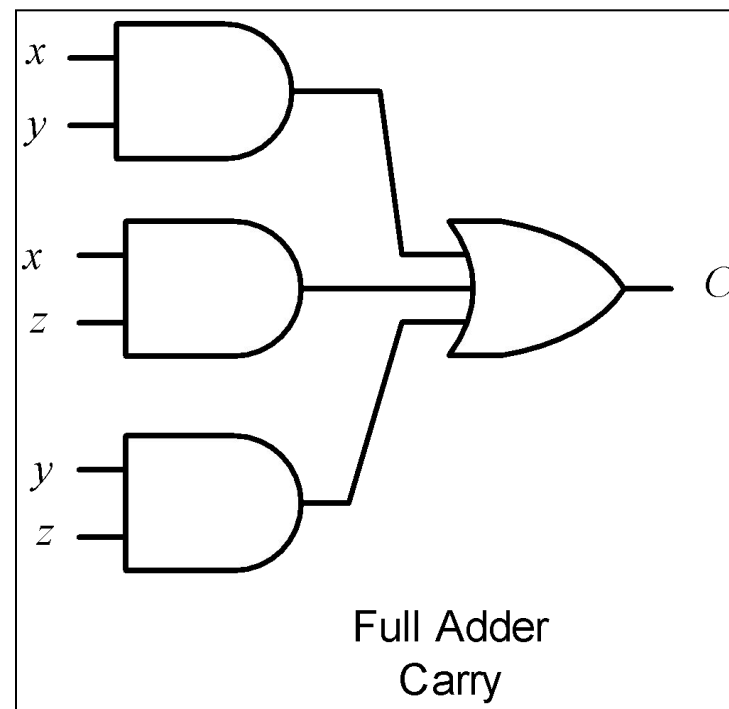
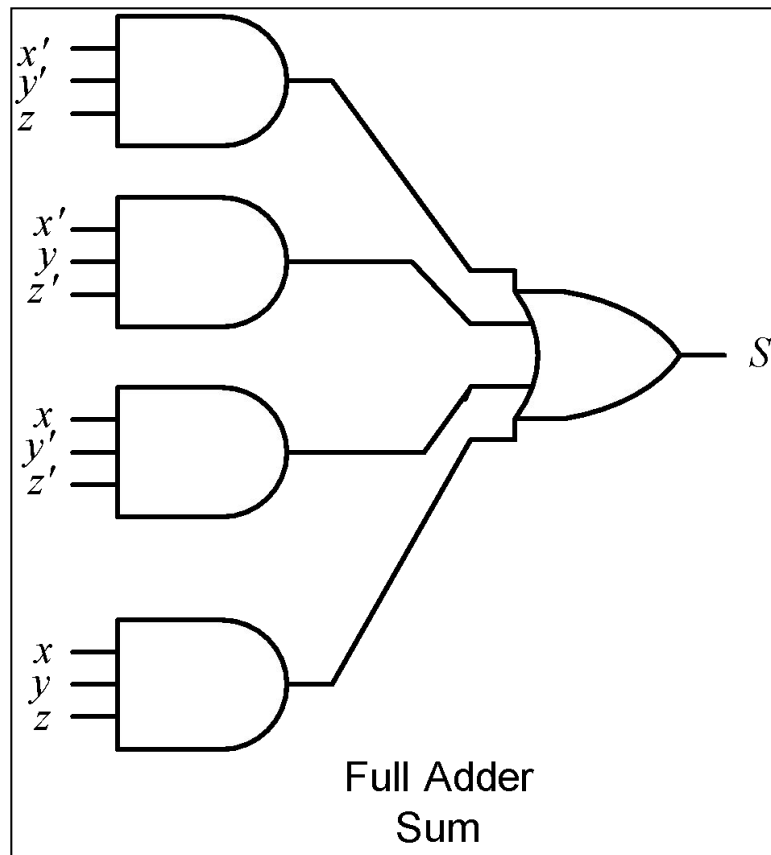
---

| <b>x</b> | <b>y</b> | <b>z</b> | <b>C</b> | <b>S</b> |
|----------|----------|----------|----------|----------|
| 0        | 0        | 0        | 0        | 0        |
| 0        | 0        | 1        | 0        | 1        |
| 0        | 1        | 0        | 0        | 1        |
| 0        | 1        | 1        | 1        | 0        |
| 1        | 0        | 0        | 0        | 1        |
| 1        | 0        | 1        | 1        | 0        |
| 1        | 1        | 0        | 1        | 0        |
| 1        | 1        | 1        | 1        | 1        |

$$\square S = x'y'z + x'yz' + xy'z' + xyz = x \oplus y \oplus z$$

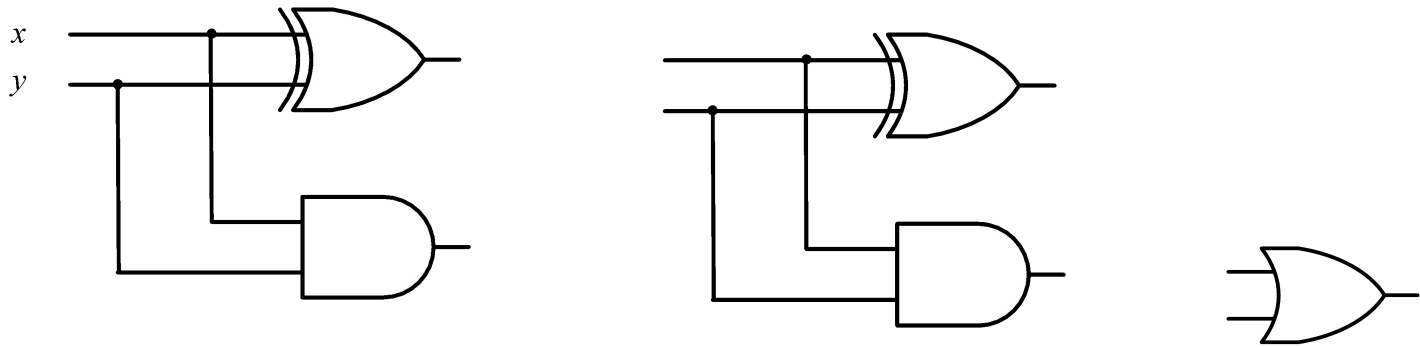
$$\square C = xy + xz + yz$$





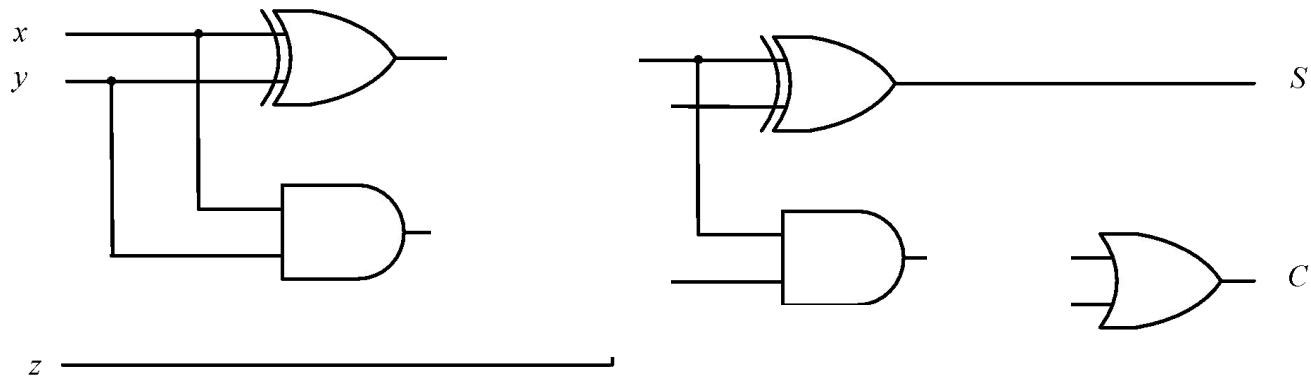
# Full Adder – with two Half Adders and an OR gate

---



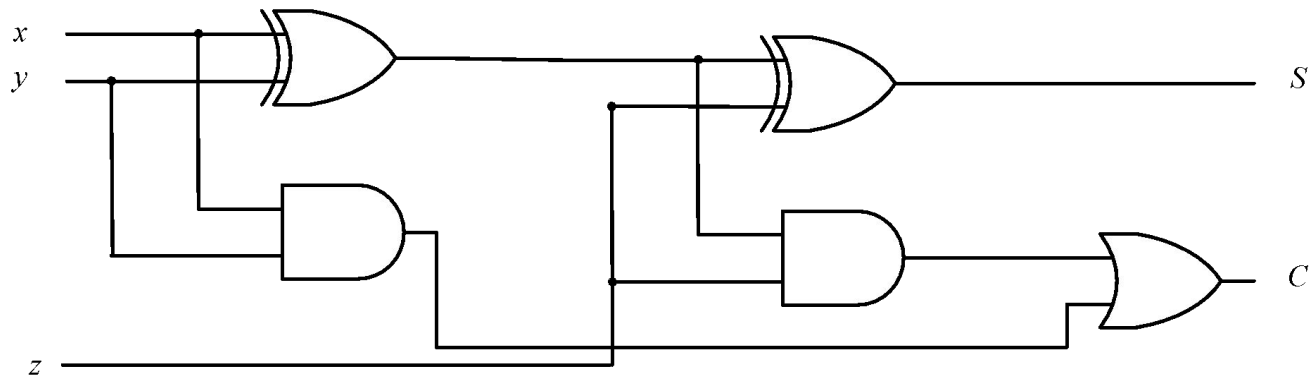
# Full Adder – with two Half Adder and an OR gate

---



# Full Adder – with two Half Adders and an OR gate

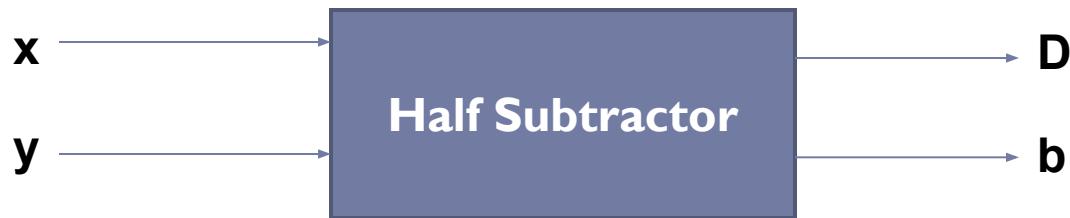
---



# Half and Full Subtractor

# Half Subtractor

- A combinational circuit that is used to get difference between two single bit numbers



| $x$ | $y$ | $D$ | $b$ |
|-----|-----|-----|-----|
| 0   | 0   | 0   | 0   |
| 0   | 1   | 1   | 1   |
| 1   | 0   | 1   | 0   |
| 1   | 1   | 0   | 0   |



# Half Subtractor

---

**K map for D**

| x \ y | 0 | 1 |
|-------|---|---|
| 0     |   | 1 |
| 1     | 1 |   |

$$x'y + xy' = x \oplus y$$

**K map for b**

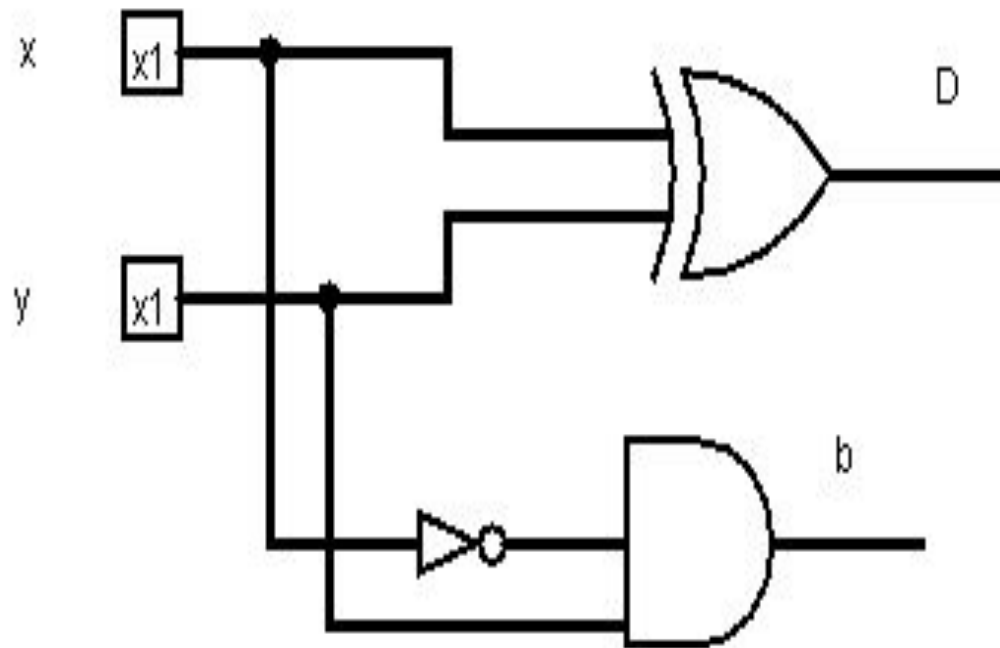
| x \ y | 0 | 1 |
|-------|---|---|
| 0     |   | 1 |
| 1     |   |   |

$$x'y$$



# Half Subtractor

---



# Full Subtractor

---

- A combinational circuit that performs subtraction of three bits

| A | B | C | D | b |
|---|---|---|---|---|
| 0 | 0 | 0 | 0 | 0 |
| 0 | 0 | 1 | 1 | 1 |
| 0 | 1 | 0 | 1 | 1 |
| 0 | 1 | 1 | 0 | 1 |
| 1 | 0 | 0 | 1 | 0 |
| 1 | 0 | 1 | 0 | 0 |
| 1 | 1 | 0 | 0 | 0 |
| 1 | 1 | 1 | 1 | 1 |



BC

|   |    |    |    |    |
|---|----|----|----|----|
|   | 00 | 01 | 11 | 10 |
| A |    |    |    |    |
| 0 |    | 1  |    | 1  |
| 1 | 1  |    | 1  |    |

$$D = AB'C' + A'B'C + ABC + A'BC'$$

$$= A \oplus B \oplus C$$

BC

|   |    |    |    |    |
|---|----|----|----|----|
|   | 00 | 01 | 11 | 10 |
| A |    |    |    |    |
| 0 |    | 1  | 1  | 1  |
| 1 |    |    | 1  |    |

$$b = BC + A'B + A'C$$

**Draw circuit diagram of Full Subtractor**



# Magnitude Comparator

# 1 Bit Magnitude Comparator

---

- A magnitude digital Comparator is a combinational circuit that **compares two digital or binary numbers** in order to find out whether one binary number is equal, less than or greater than the other binary number



# 1 Bit Magnitude Comparator



| <i><b>A</b></i> | <i><b>B</b></i> | <i><b>A&gt;B</b></i> | <i><b>A=B</b></i> | <i><b>A&lt;B</b></i> |
|-----------------|-----------------|----------------------|-------------------|----------------------|
| 0               | 0               | 0                    | 1                 | 0                    |
| 0               | 1               | 0                    | 0                 | 1                    |
| 1               | 0               | 1                    | 0                 | 0                    |
| 1               | 1               | 0                    | 1                 | 0                    |



# 1 Bit Magnitude Comparator

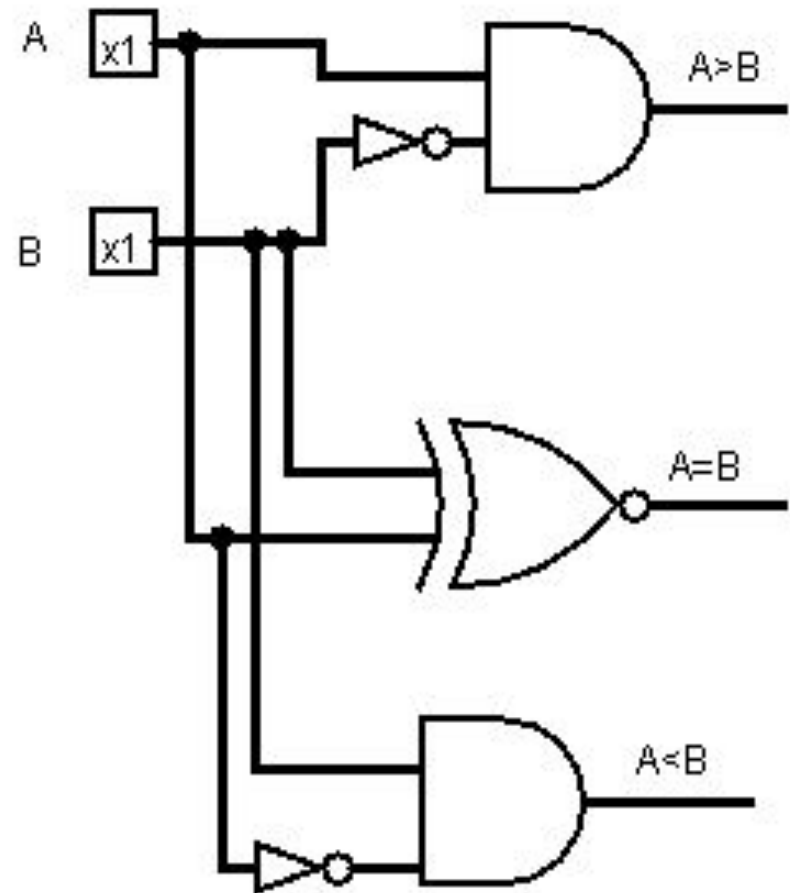
After simplification through, following expressions will be obtained

Single bit comparison

$$A > B \rightarrow \bar{A}B$$

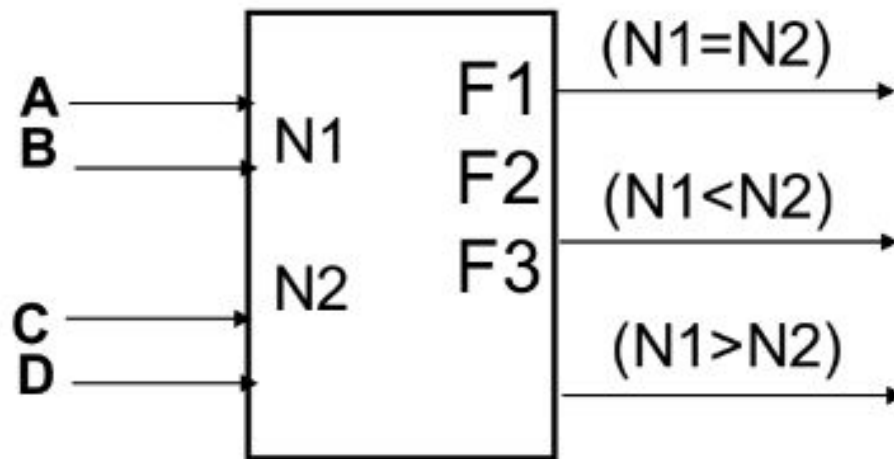
$$A < B \rightarrow A\bar{B}$$

$$A = B \rightarrow \overline{A \oplus B} = A'B' + AB$$



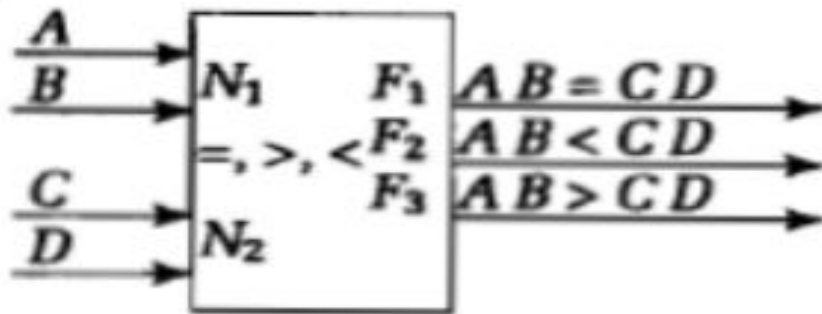
## 2-Bit Magnitude Comparator

- Design a circuit that has two 2-bit numbers  $N1$  and  $N2$  as inputs, and generates three outputs to indicate if  $N1 < N2$ ,  $N1 = N2$  and  $N1 > N2$ .



$N1 = AB$   
 $N2 = CD$

# 2-Bit Magnitude Comparator



| $A$ | $B$ | $C$ | $D$ | $F_1$ | $F_2$ | $F_3$ |
|-----|-----|-----|-----|-------|-------|-------|
| 0   | 0   | 0   | 0   | 1     | 0     | 0     |
|     |     | 0   | 1   | 0     | 1     | 0     |
|     |     | 1   | 0   | 0     | 1     | 0     |
|     |     | 1   | 1   | 0     | 1     | 0     |
| 0   | 1   | 0   | 0   | 0     | 0     | 1     |
|     |     | 0   | 1   | 1     | 0     | 0     |
|     |     | 1   | 0   | 0     | 1     | 0     |
|     |     | 1   | 1   | 0     | 1     | 0     |
| 1   | 0   | 0   | 0   | 0     | 0     | 1     |
|     |     | 0   | 1   | 0     | 0     | 1     |
|     |     | 1   | 0   | 1     | 0     | 0     |
|     |     | 1   | 1   | 0     | 1     | 0     |
| 1   | 1   | 0   | 0   | 0     | 0     | 1     |
|     |     | 0   | 1   | 0     | 0     | 1     |
|     |     | 1   | 0   | 0     | 0     | 1     |
|     |     | 1   | 1   | 1     | 0     | 0     |

# 2-Bit Magnitude Comparator

| AB \ CD |    | A  |    |    |    |
|---------|----|----|----|----|----|
|         |    | 00 | 01 | 11 | 10 |
| C       | 00 | 1  | 0  | 0  | 0  |
|         | 01 | 0  | 1  | 0  | 0  |
|         | 11 | 0  | 0  | 1  | 0  |
|         | 10 | 0  | 0  | 0  | 1  |

K-map for  $F_1$

| AB \ CD |    | A  |    |    |    |
|---------|----|----|----|----|----|
|         |    | 00 | 01 | 11 | 10 |
| C       | 00 | 0  | 0  | 0  | 0  |
|         | 01 | 1  | 0  | 0  | 0  |
|         | 11 | 1  | 1  | 0  | 1  |
|         | 10 | 1  | 1  | 0  | 0  |

K-map for  $F_2$

| AB \ CD |    | A  |    |    |    |
|---------|----|----|----|----|----|
|         |    | 00 | 01 | 11 | 10 |
| C       | 00 | 0  | 1  | 1  | 1  |
|         | 01 | 0  | 0  | 1  | 1  |
|         | 11 | 0  | 0  | 0  | 0  |
|         | 10 | 0  | 0  | 1  | 0  |

K-map for  $F_3$

$$F_1 = \overline{A}\overline{B}\overline{C}\overline{D} + \overline{A}B\overline{C}\overline{D} + A\overline{B}\overline{C}\overline{D} + ABCD$$

$$F_1 = \overline{A}\overline{C}(\overline{B}\overline{D} + BD) + AC(\overline{B}\overline{D} + BD)$$

$$= (\overline{A}\overline{C} + AC)(\overline{B}\overline{D} + BD)$$

$$= (\overline{A \oplus C})(\overline{B \oplus D})$$

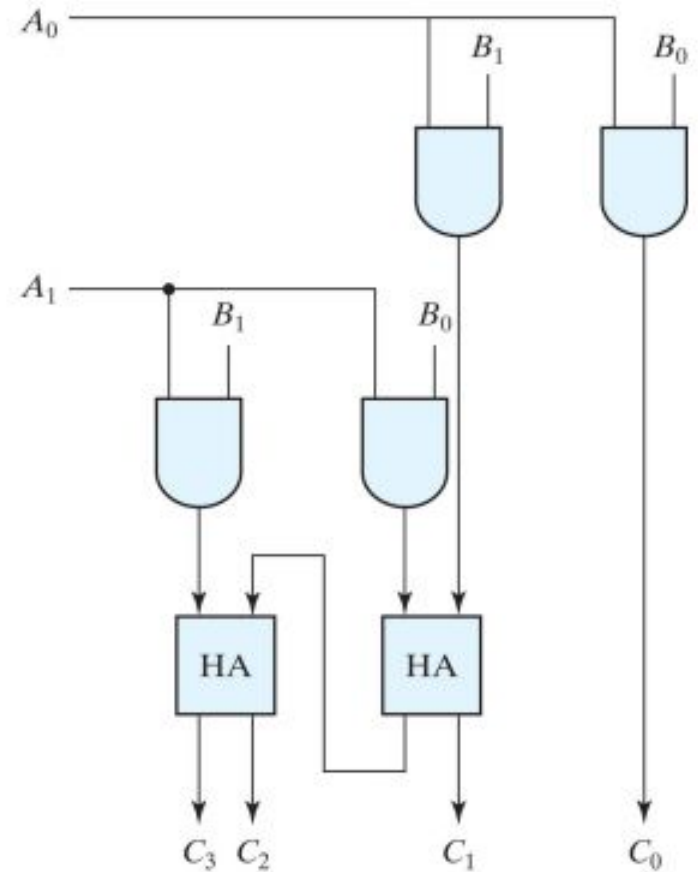
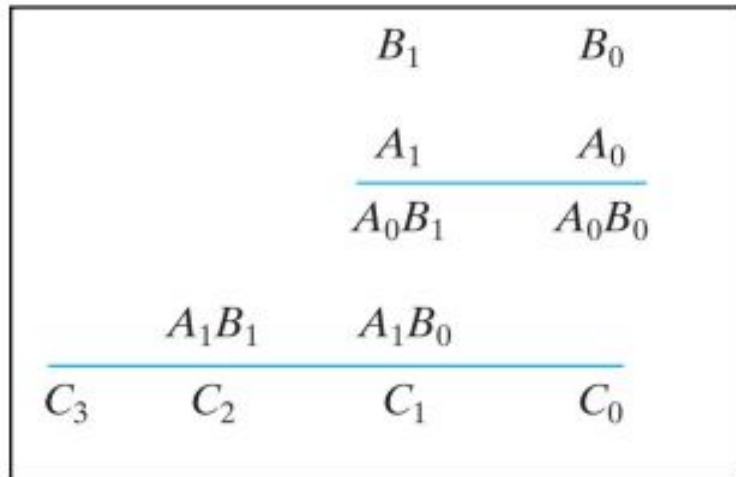
$$F_2 = \overline{A}\overline{B}D + \overline{B}CD + \overline{A}C$$

$$F_3 = A\overline{C} + AB\overline{D} + B\overline{C}\overline{D}$$



# Binary Multiplier

# 2 bit by 2 bit binary multiplier



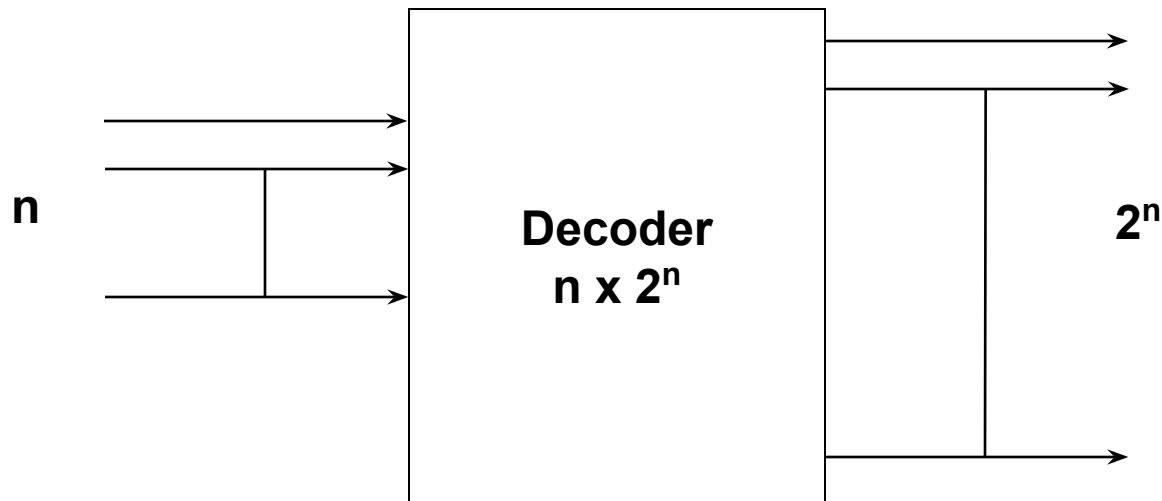


Decoder

# Decoder

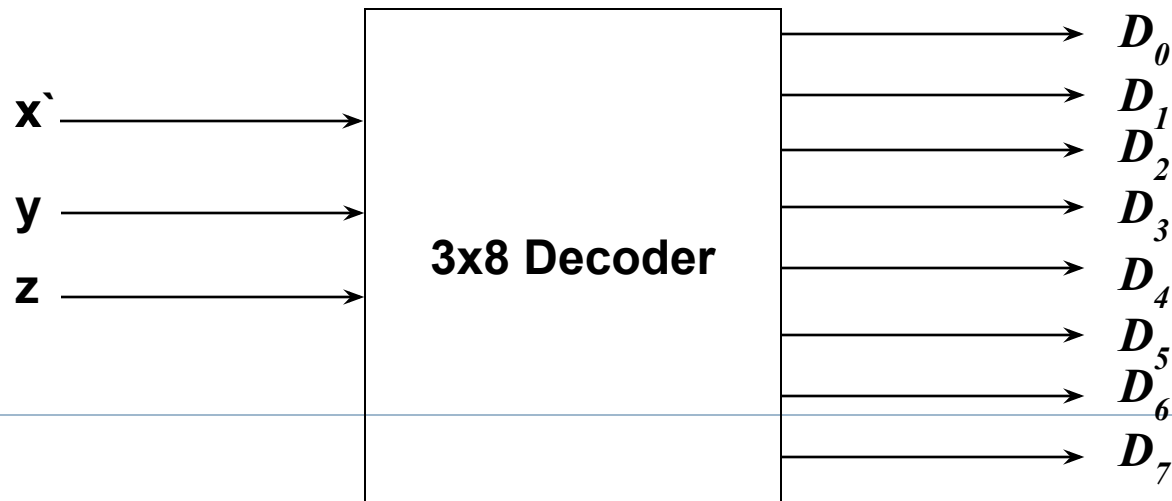
---

- A combinational circuit that converts binary information from  $n$  input lines to a maximum of  $2^n$  *unique* output lines.



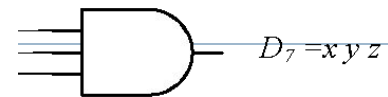
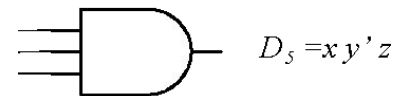
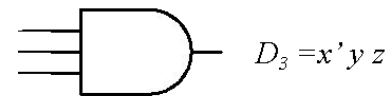
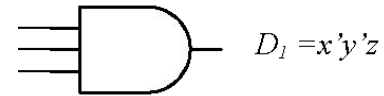
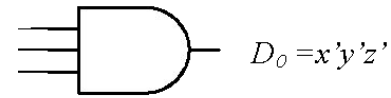
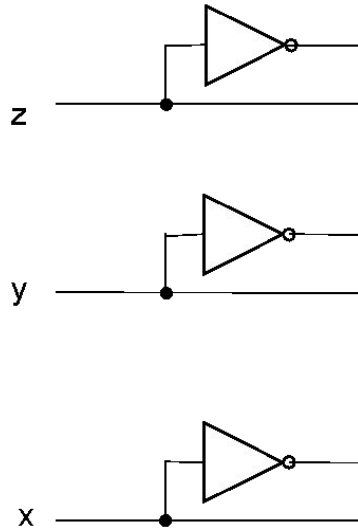
# Truth Table for 3-to-8 Line Decoder

| Inputs |   |   | Outputs |    |    |    |    |    |    |    |
|--------|---|---|---------|----|----|----|----|----|----|----|
| x      | y | z | D0      | D1 | D2 | D3 | D4 | D5 | D6 | D7 |
| 0      | 0 | 0 |         |    |    |    |    |    |    |    |
| 0      | 0 | 1 |         |    |    |    |    |    |    |    |
| 0      | 1 | 0 |         |    |    |    |    |    |    |    |
| 0      | 1 | 1 |         |    |    |    |    |    |    |    |
| 1      | 0 | 0 |         |    |    |    |    |    |    |    |
| 1      | 0 | 1 |         |    |    |    |    |    |    |    |
| 1      | 1 | 0 |         |    |    |    |    |    |    |    |
| 1      | 1 | 1 |         |    |    |    |    |    |    |    |



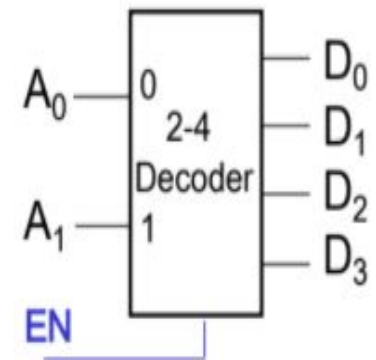
# 3 to 8 Decoder

---



# 2-4 Line NAND Decoder with Enable Input

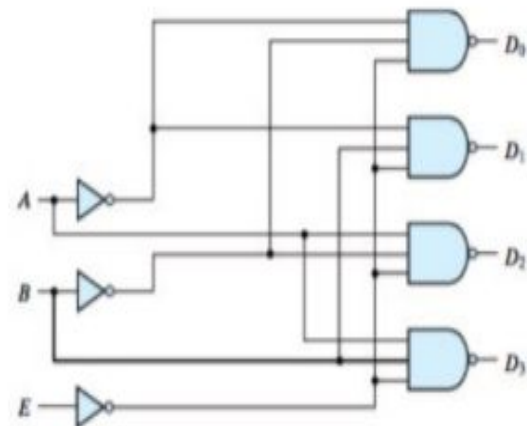
- Extra input EN:
  - If EN=1: decoder is disabled
  - If EN=0: decoder is enabled [active low outputs]



- Truth Table and Logic Circuit [using NAND Gates]

Active low outputs!!

| $E$ | $A$ | $B$ | $D_0$ | $D_1$ | $D_2$ | $D_3$ |
|-----|-----|-----|-------|-------|-------|-------|
| 1   | X   | X   | 1     | 1     | 1     | 1     |
| 0   | 0   | 0   | 0     | 1     | 1     | 1     |
| 0   | 0   | 1   | 1     | 0     | 1     | 1     |
| 0   | 1   | 0   | 1     | 1     | 0     | 1     |
| 0   | 1   | 1   | 1     | 1     | 1     | 0     |



# Implementation of Boolean Function with Decoder

---

- Express the function in sum of minterms form
- Select a decoder with number of inputs = number of variables in the function
- Connect the variables to the input in order
- Connect the outputs equivalent to minterms (in sum of minterms form) to the input of OR gate. Function is implemented.



# Implementation of Boolean Function with Decoder

---

- Implement the following function using a decoder:

$$S(x,y,z) = x'y'z + x'yz' + xy'z' + xyz$$



# Implementation of Boolean Function with Decoder

---

▣ Implement the following function using a decoder:

▶  $F(A, B, C) = A'B'C + AB' + BC$

# Implementation of Boolean Function with Decoder

---

▣ Implement the following function using a decoder:

▶  $F(A, B, C) = A'B'C + AB' + BC$

# Implementation of Boolean Function with Decoder

---

- Using decoder and external gates design combinational circuit defined by following Boolean functions

$$F1(x,y,z)=x'y'z'+xz+yz$$

$$F2(x,y,z)=xy'z'+x'y$$

$$F3(x,y,z)=x'y'z+xy$$



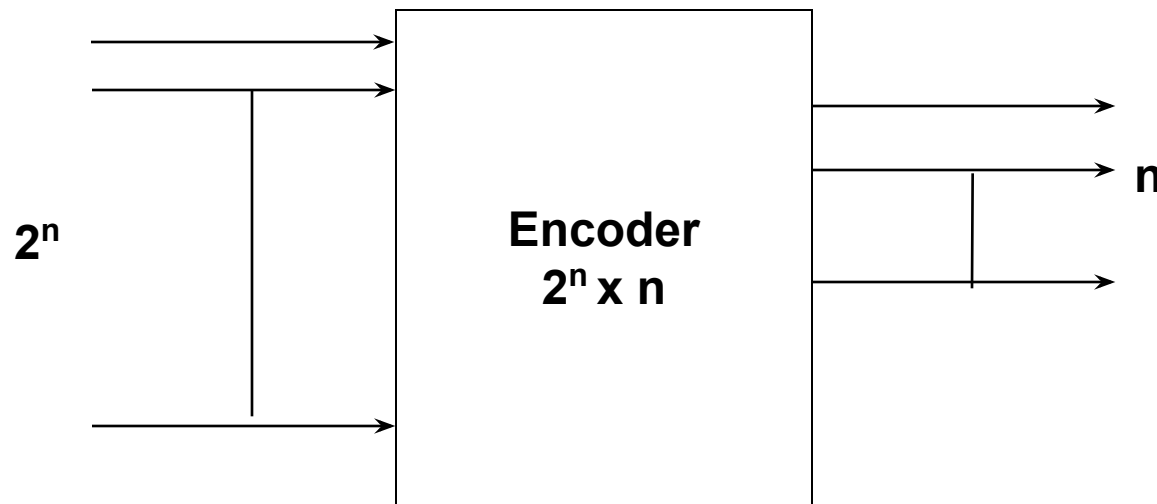


Encoder

# ENCODERS

---

- Encoder is a digital circuit that performs inverse operation of the decoder.
- An encoder has  $2^n$  inputs lines and  $n$  output lines.



# ENCODERS

---

## TYPES OF ENCODERS:

- HEX/OCTAL/QUAD... TO BINARY ENCODER
- PRIORITY ENCODER



# Truth Table - Octal To Binary Encoder

| Inputs |    |    |    |    |    |    |    | Outputs |   |   |
|--------|----|----|----|----|----|----|----|---------|---|---|
| D0     | D1 | D2 | D3 | D4 | D5 | D6 | D7 | x       | y | z |

$$z = D_1 + D_3 + D_5 + D_7$$

$$y = D_2 + D_3 + D_6 + D_7$$

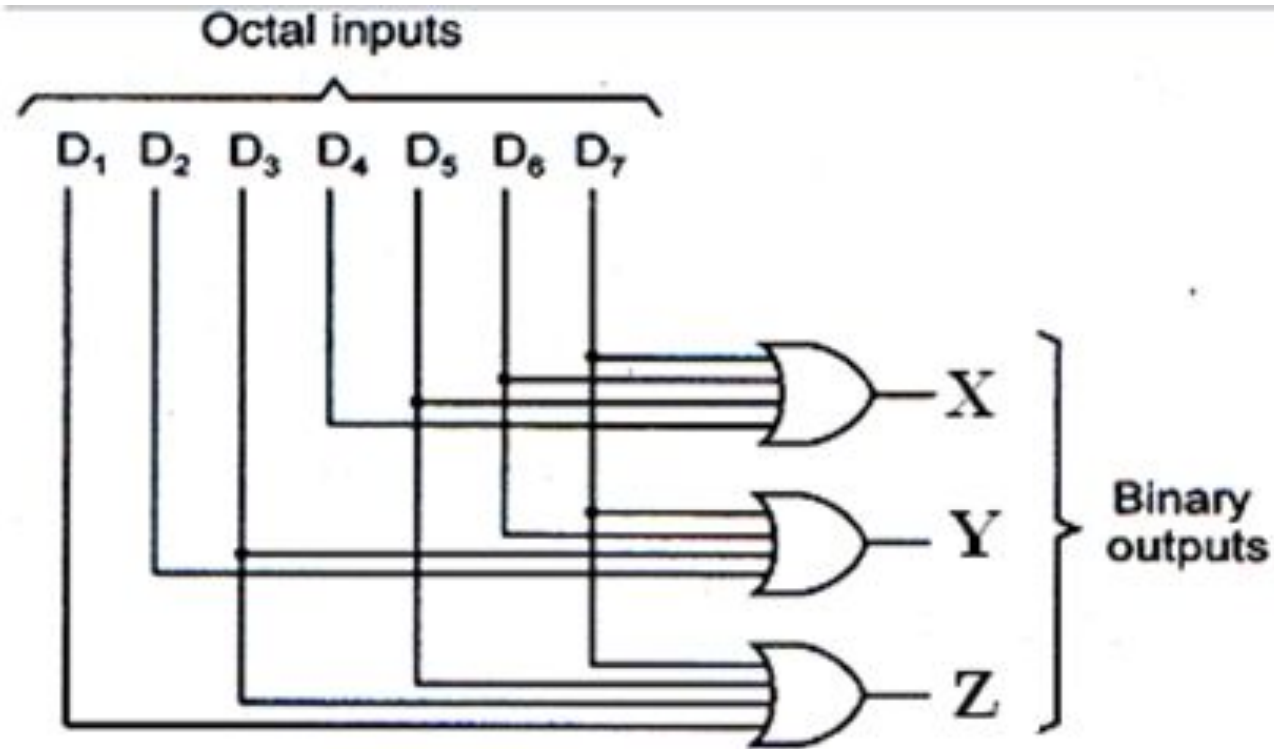
$$x = D_4 + D_5 + D_6 + D_7$$

# TRUTH TABLE OF AN OCTAL TO BINARY ENCODER

*Truth Table of an Octal-to-Binary Encoder*

| Inputs |       |       |       |       |       |       |       | Outputs |     |     |
|--------|-------|-------|-------|-------|-------|-------|-------|---------|-----|-----|
| $D_0$  | $D_1$ | $D_2$ | $D_3$ | $D_4$ | $D_5$ | $D_6$ | $D_7$ | $x$     | $y$ | $z$ |
| 1      | 0     | 0     | 0     | 0     | 0     | 0     | 0     | 0       | 0   | 0   |
| 0      | 1     | 0     | 0     | 0     | 0     | 0     | 0     | 0       | 0   | 1   |
| 0      | 0     | 1     | 0     | 0     | 0     | 0     | 0     | 0       | 1   | 0   |
| 0      | 0     | 0     | 1     | 0     | 0     | 0     | 0     | 0       | 1   | 1   |
| 0      | 0     | 0     | 0     | 1     | 0     | 0     | 0     | 1       | 0   | 0   |
| 0      | 0     | 0     | 0     | 0     | 1     | 0     | 0     | 1       | 0   | 1   |
| 0      | 0     | 0     | 0     | 0     | 0     | 1     | 0     | 1       | 1   | 0   |
| 0      | 0     | 0     | 0     | 0     | 0     | 0     | 1     | 1       | 1   | 1   |

# OCTAL TO BINARY ENCODER





# Priority Encoder

# PRIORITY ENCODER

---

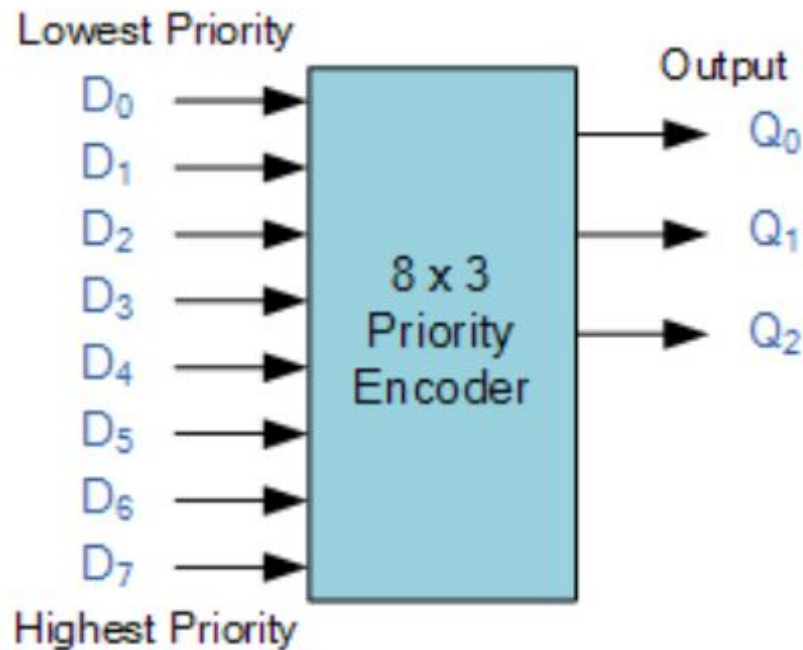
- A priority encoder is an encoder that includes the priority function.
- If two or more inputs are equal to 1 at the same time, the input with higher priority will take precedence.



# PRIORITY ENCODER

---

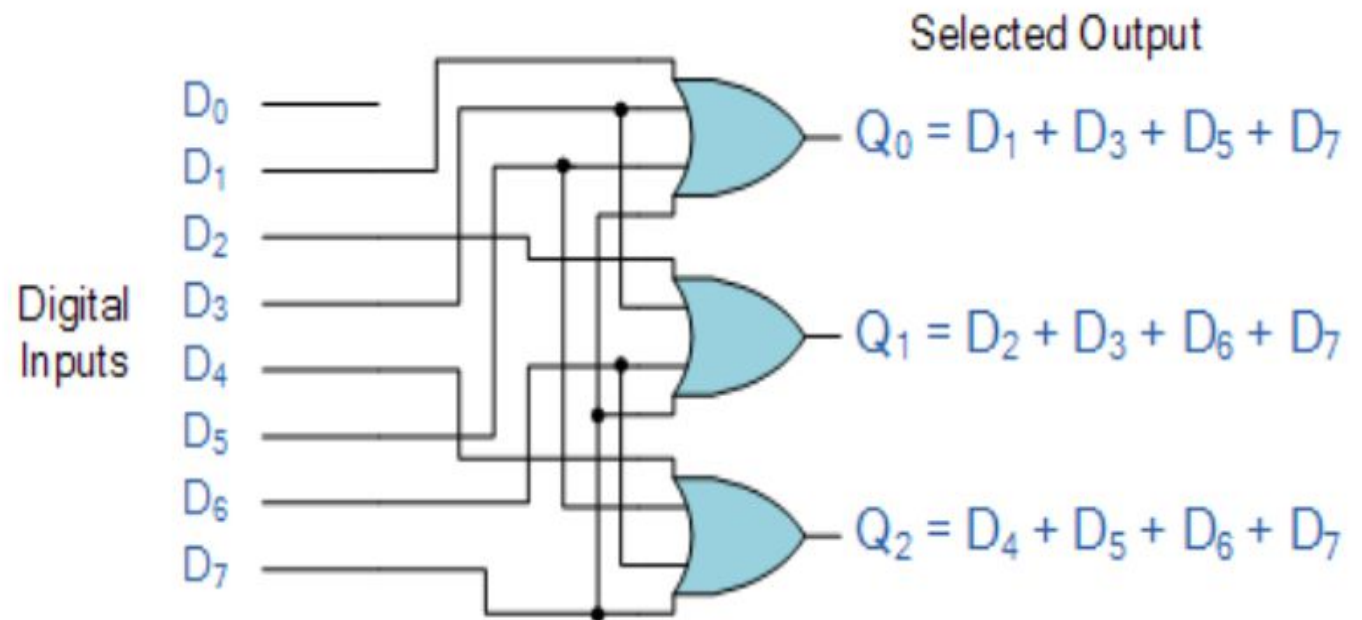
## 8-to-3 Bit Priority Encoder





# PRIORITY ENCODER

## Digital Encoder using Logic Gates



# Problem

---

- Design a priority encoder with 4 inputs ( $D_0, D_1, D_2, D_3$ ) and 3 outputs ( $A_0, A_1, V$ ) where most significant bit has highest priority



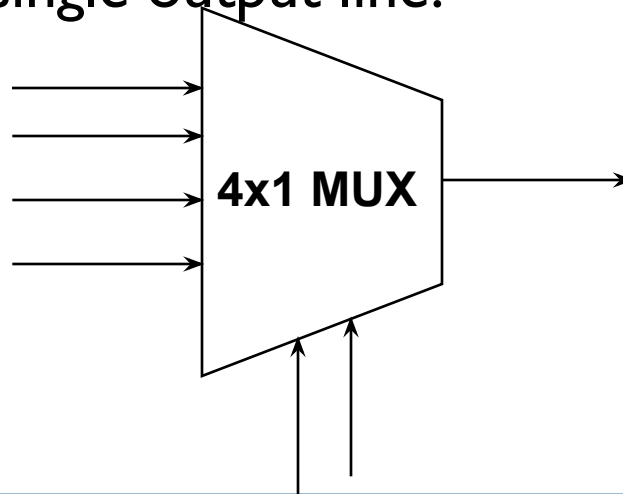


# Multiplexer

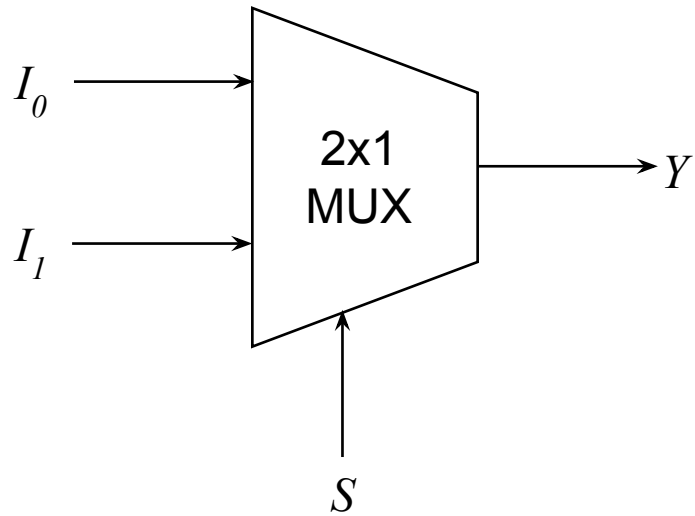
# Multiplexer

---

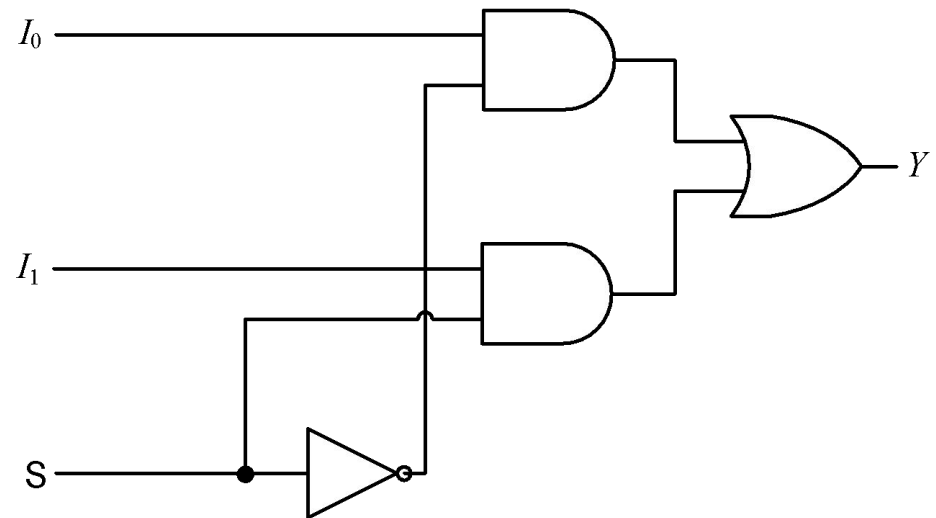
- A combinational circuit that selects binary information from one of many input lines and...
- Multiplexer is a combinational circuit used to select only one input among several inputs based on selection lines
- MUX can have  $2^n$  inputs,  $n$  selection lines and only one output
- directs it to a single output line.



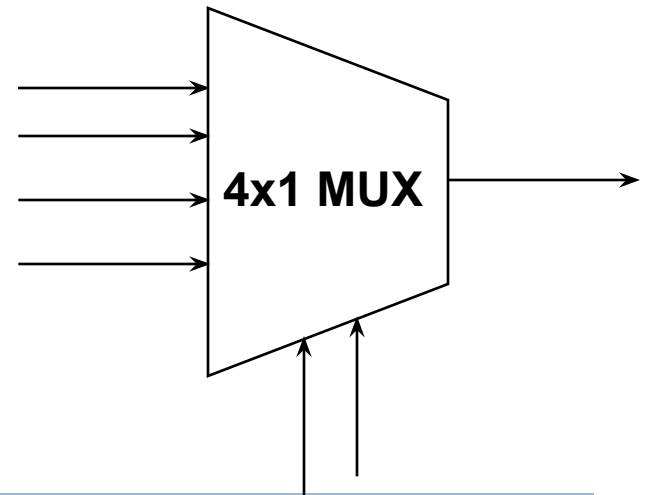
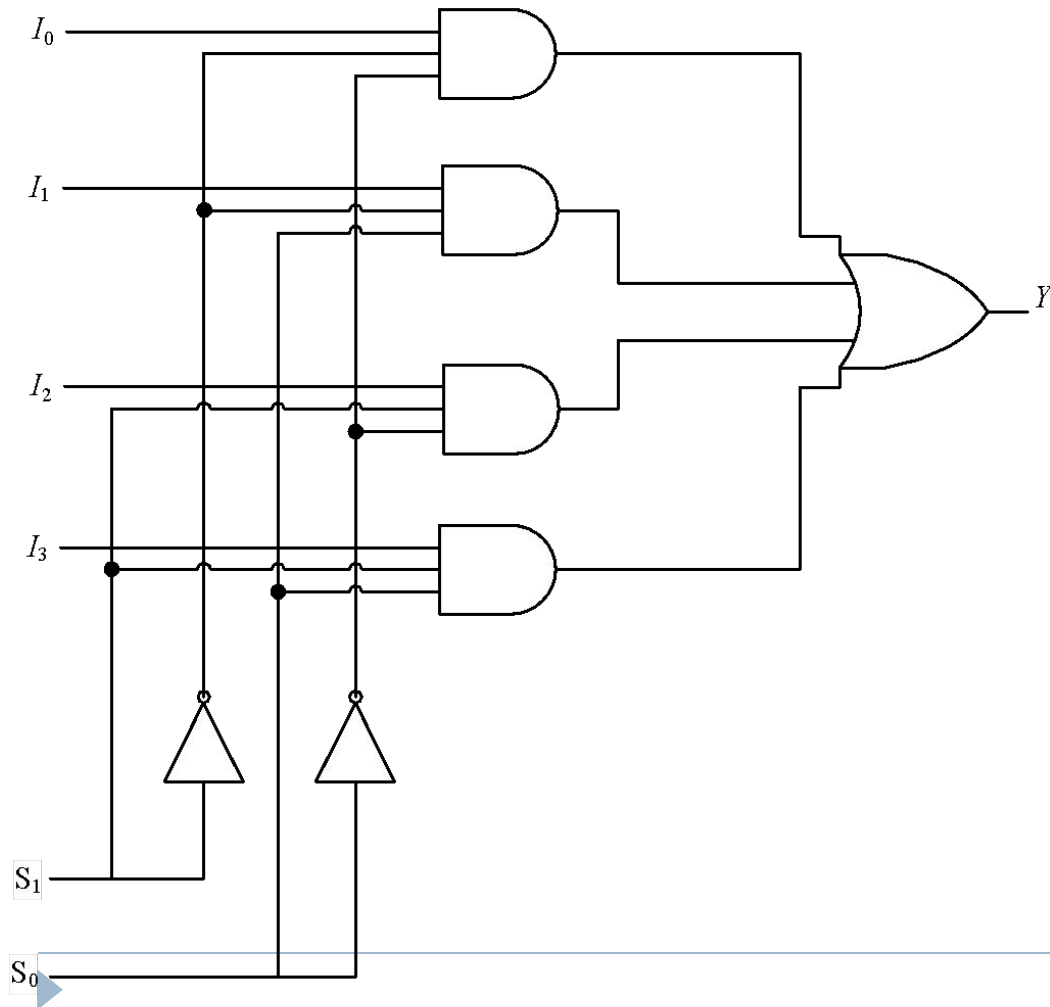
# 2x1 MUX



| $S$ | $Y$   |
|-----|-------|
| 0   | $I_0$ |
| 1   | $I_1$ |



# 4x1 MUX



# MUX Implementation

---

- Draw the TT of the Boolean function
- Select a MUX with number of select-lines = number of input variables in the Boolean function
- Connect the input variables to the select-lines in order
- Connect the input lines of the MUX with 1s and 0s as per the TT i.e.  $I_0$  with output value of  $m_0$ ,  $I_1$  to  $m_1$  value, and so on.
- The function is implemented.



# MUX Implementation ()

---

- Implement  $F(a,b,c) = \Sigma(1,2,3,6,7)$  using 8x1 MUX



## MUX( $n-1$ ) Implementation ()

---

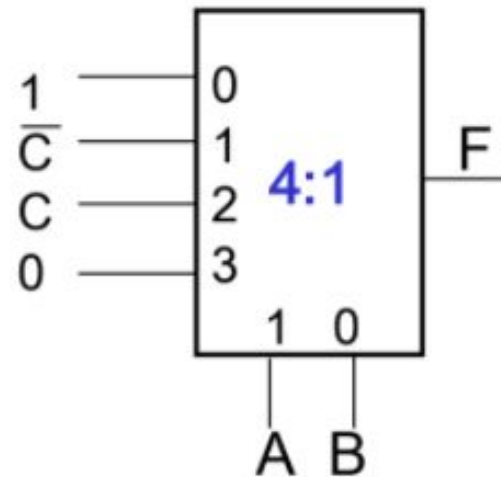
- Use a multiplexer with  $n-1$  selection inputs and  $2^{n-1}$  data inputs.
- List the truth table.
- Apply the first  $n-1$  variables to the selection inputs of multiplexer.
- For each combination evaluate the output as a function of the last variable.
- The function can be 0, 1 the variable or the complement of the variable.



# MUX(n-1) Implementation ()

- Implement the function  $F(A,B,C) = \sum(0,1,2,5)$  using a 4:1 Mux.

| A | B | C | F |                |
|---|---|---|---|----------------|
| 0 | 0 | 0 | 1 | 1              |
| 0 | 0 | 1 | 1 |                |
| 0 | 1 | 0 | 1 | $\overline{C}$ |
| 0 | 1 | 1 | 0 |                |
| 1 | 0 | 0 | 0 | C              |
| 1 | 0 | 1 | 1 |                |
| 1 | 1 | 0 | 0 | 0              |
| 1 | 1 | 1 | 0 |                |



# MUX (n-1) Implementation

---

□  $F(A,B,C,D) = \Sigma(1,3,4,11,12,13,14,15)$  using 8:1 MUX

| A | B | C | D | F |
|---|---|---|---|---|
| 0 | 0 | 0 | 0 | 0 |
| 0 | 0 | 0 | 1 | 1 |
| 0 | 0 | 1 | 0 | 0 |
| 0 | 0 | 1 | 1 | 1 |
| 0 | 1 | 0 | 0 | 1 |
| 0 | 1 | 0 | 1 | 0 |
| 0 | 1 | 1 | 0 | 0 |
| 0 | 1 | 1 | 1 | 0 |
| 1 | 0 | 0 | 0 | 0 |
| 1 | 0 | 0 | 1 | 0 |
| 1 | 0 | 1 | 0 | 0 |
| 1 | 0 | 1 | 1 | 1 |
| 1 | 1 | 0 | 0 | 1 |
| 1 | 1 | 0 | 1 | 1 |
| 1 | 1 | 1 | 0 | 1 |
| 1 | 1 | 1 | 1 | 1 |



# MUX (n-1) Implementation

---

□  $F(A,B,C,D) = \Sigma(1,3,4,11,12,13,14,15)$

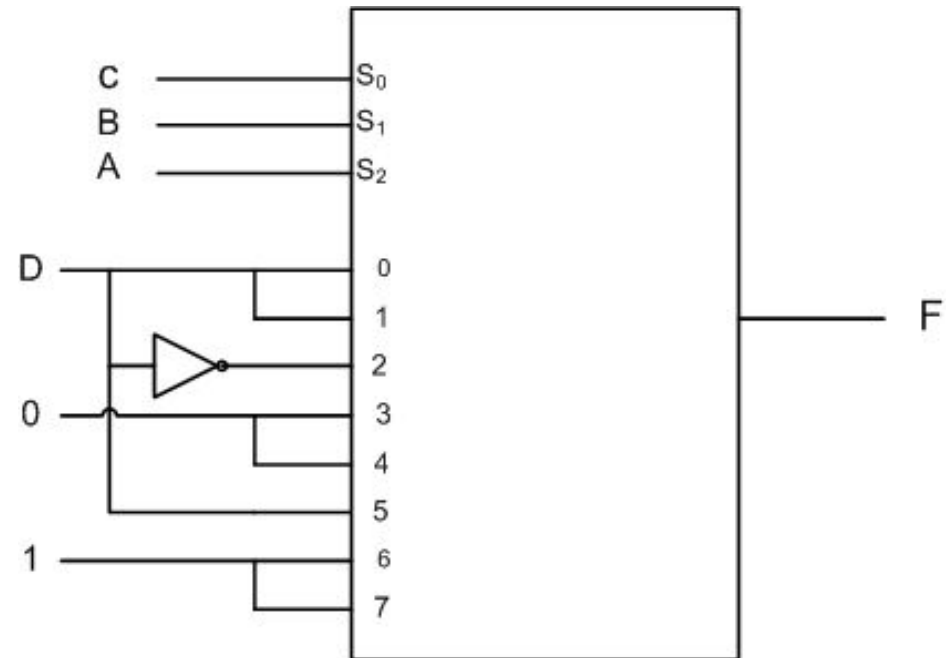
| A | B | C | D | F |
|---|---|---|---|---|
|   |   |   | 0 | 0 |
|   |   |   | 1 | 1 |
| - |   |   | 0 | 0 |
|   |   |   | 1 | 1 |
| - |   |   | 0 | 1 |
|   |   |   | 1 | 0 |
| - |   |   | 0 | 0 |
|   |   |   | 1 | 0 |
| - |   |   | 0 | 0 |
|   |   |   | 1 | 0 |
| - |   |   | 0 | 0 |
|   |   |   | 1 | 1 |
| - |   |   | 0 | 1 |
|   |   |   | 1 | 1 |
| - |   |   | 0 | 1 |
|   |   |   | 1 | 1 |



# MUX (n-1) Implementation

□  $F(A,B,C,D) = \Sigma(1,3,4,11,12,13,14,15)$

| A | B | C | D | F |
|---|---|---|---|---|
| 0 | 0 | 0 | 0 | 0 |
| 0 | 0 | 0 | 1 | 1 |
| 0 | 0 | 1 | 0 | 0 |
| 0 | 0 | 1 | 1 | 1 |
| 0 | 1 | 0 | 0 | 1 |
| 0 | 1 | 0 | 1 | 0 |
| 0 | 1 | 1 | 0 | 0 |
| 0 | 1 | 1 | 1 | 0 |
| 1 | 0 | 0 | 0 | 0 |
| 1 | 0 | 0 | 1 | 0 |
| 1 | 0 | 1 | 0 | 0 |
| 1 | 0 | 1 | 1 | 1 |
| 1 | 1 | 0 | 0 | 1 |
| 1 | 1 | 0 | 1 | 1 |
| 1 | 1 | 1 | 0 | 1 |
| 1 | 1 | 1 | 1 | 1 |





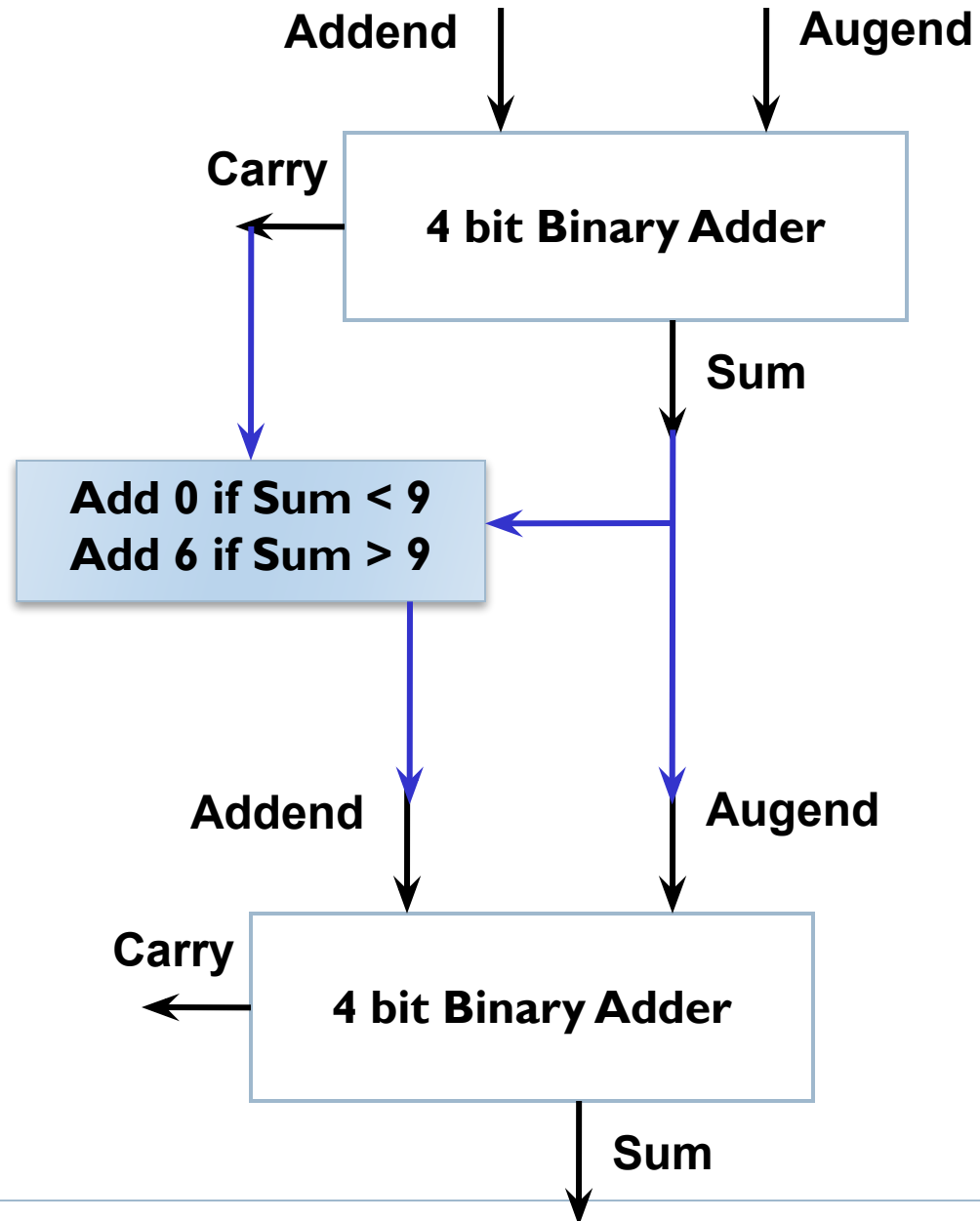
# Selected Problems

# BCD Adder

---

- Using 4-bit binary adder to get BCD addition
- 4-bit binary adder will give Binary Sum.
- Is Binary Sum = BCD Sum?
  
- How to convert the Binary Sum into BCD Sum?





# BCD Adder

| Binary Sum |    |    |    |    | BCD Sum |    |    |    |    | Decimal |
|------------|----|----|----|----|---------|----|----|----|----|---------|
| K          | Z8 | Z4 | Z2 | Z1 | C       | S8 | S4 | S2 | S1 |         |
| 0          | 0  | 0  | 0  | 0  | 0       | 0  | 0  | 0  | 0  | 0       |
| 0          | 0  | 0  | 0  | 1  | 0       | 0  | 0  | 0  | 1  | 1       |
| 0          | 0  | 0  | 1  | 0  | 0       | 0  | 0  | 1  | 0  | 2       |
| 0          | 0  | 0  | 1  | 1  | 0       | 0  | 0  | 1  | 1  | 3       |
| 0          | 0  | 1  | 0  | 0  | 0       | 0  | 1  | 0  | 0  | 4       |
| 0          | 0  | 1  | 0  | 1  | 0       | 0  | 1  | 0  | 1  | 5       |
| 0          | 0  | 1  | 1  | 0  | 0       | 0  | 1  | 1  | 0  | 6       |
| 0          | 0  | 1  | 1  | 1  | 0       | 0  | 1  | 1  | 1  | 7       |
| 0          | 1  | 0  | 0  | 0  | 0       | 1  | 0  | 0  | 0  | 8       |
| 0          | 1  | 0  | 0  | 1  | 0       | 1  | 0  | 0  | 1  | 9       |
| 0          | 1  | 0  | 1  | 0  | 1       | 0  | 0  | 0  | 0  | 10      |
| 0          | 1  | 0  | 1  | 1  | 1       | 0  | 0  | 0  | 1  | 11      |
| 0          | 1  | 1  | 0  | 0  | 1       | 0  | 0  | 1  | 0  | 12      |
| 0          | 1  | 1  | 0  | 1  | 1       | 0  | 0  | 1  | 1  | 13      |
| 0          | 1  | 1  | 1  | 0  | 1       | 0  | 1  | 0  | 0  | 14      |
| 0          | 1  | 1  | 1  | 1  | 1       | 0  | 1  | 0  | 1  | 15      |
| 1          | 0  | 0  | 0  | 0  | 1       | 0  | 1  | 1  | 0  | 16      |
| 1          | 0  | 0  | 0  | 1  | 1       | 0  | 1  | 1  | 1  | 17      |
| 1          | 0  | 0  | 1  | 0  | 1       | 1  | 0  | 0  | 0  | 18      |
| 1          | 0  | 0  | 1  | 1  | 1       | 1  | 0  | 0  | 1  | 19      |

# BCD Adder:

$$F(K, Z_8, Z_4, Z_2, Z_1)$$

$K = 0$

| $Z_8 Z_4 \backslash Z_2 Z_1$ | 00 | 01 | 11 | 10 |
|------------------------------|----|----|----|----|
| 00                           |    |    |    |    |
| 01                           |    |    |    |    |
| 11                           | 1  | 1  | 1  | 1  |
| 10                           |    |    | 1  | 1  |

$K = 1$

| $Z_8 Z_4 \backslash Z_2 Z_1$ | 00 | 01 | 11 | 10 |
|------------------------------|----|----|----|----|
| 00                           | 1  | 1  | 1  | 1  |
| 01                           | x  | x  | x  | x  |
| 11                           | x  | x  | x  | x  |
| 10                           | x  | x  | x  | x  |

$$C = K + Z_8 Z_4 + Z_8 Z_2$$

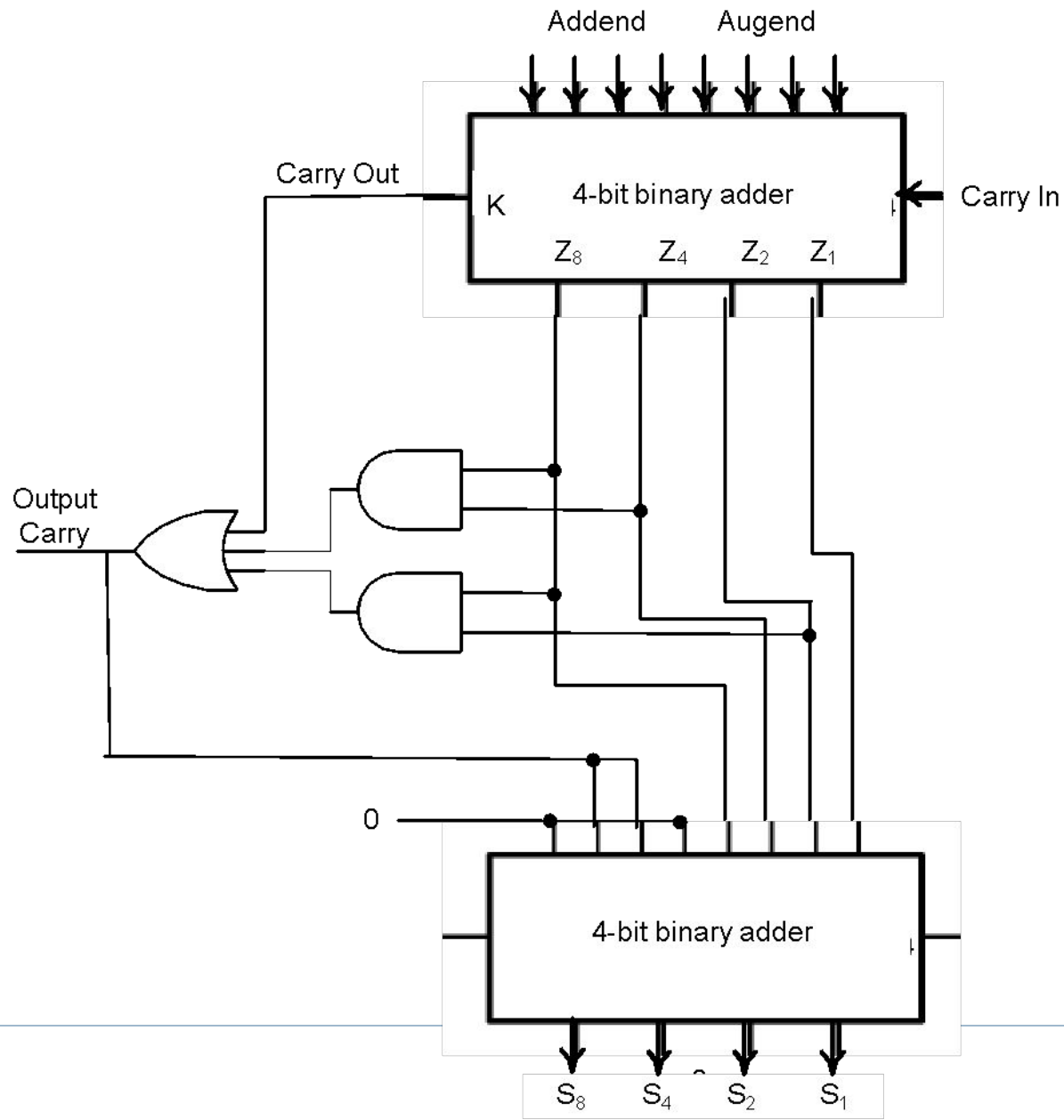
# BCD Adder

---

$$\square C = K + Z_8 Z_4 + Z_8 Z_2$$



# BCD Adder



4-4 Design a combinational circuit with three inputs and one output. The output is 1 when the binary value of the inputs is less than 3. The output is 0 otherwise.



4-6 The majority circuit is a combinational circuit whose output is equal to 1 if the input variables have more 1's than 0's. The output is 0 otherwise. Design a 3 bit majority circuit.



4-28 A combinational circuit is defined by the following Boolean functions:

$$F_1 = x'y'z' + xz$$

$$F_2 = xy'z' + x'y$$

$$F_3 = x'y'z + xy$$

Design the circuit with decoder and external gates.



4.29 Realize the following Boolean expression using an 8x1 multiplexer.

$$Y(A,B,C) = A'B'C + A'BC' + AB'C + ABC$$



4.34 An 8x1 multiplexer has inputs A, B, and C connected to the selection inputs  $S_2$ ,  $S_1$ , and  $S_0$ , respectively. The data inputs  $I_0$  through  $I_7$  are as follows:  $I_1=I_2=I_7=0$ ;  $I_3=I_5=1$ ;  $I_0=I_4=D$ ; and  $I_6=D'$ . Determine the Boolean function that the multiplexer implements.



- Design a half-subtractor circuit with inputs  $x$  and  $y$  and outputs  $D$  and  $B$ . The circuit subtracts  $x - y$ , and places the difference in  $D$  and the borrow in  $B$ .

| $x$ | $y$ | $B$ | $D$ |
|-----|-----|-----|-----|
| 0   | 0   | 0   | 0   |
| 0   | 1   | 1   | 1   |
| 1   | 0   | 0   | 1   |
| 1   | 1   | 0   | 0   |



**Design a full-subtractor circuit with inputs x and y and outputs D and B. The circuit subtracts  $x - y - z$ , and places the difference in D and the borrow in B.**

| x | y | z | B | D |
|---|---|---|---|---|
| 0 | 0 | 0 | 0 | 0 |
| 0 | 0 | 1 | 1 | 1 |
| 0 | 1 | 0 | 1 | 1 |
| 0 | 1 | 1 | 0 | 1 |
| 1 | 0 | 0 | 1 | 0 |
| 1 | 0 | 1 | 0 | 0 |
| 1 | 1 | 0 | 0 | 0 |
| 1 | 1 | 1 | 1 | 1 |



**Thank you**

